

Table 14 - 3. Performance of models

Class Name	Size (MB)	Top-1 Acc	Top-5 Acc	Params	CPU (ms)	GPU (ms)
MobileNetV2	14	71.3%	90.1%	3.5M	25.9	3.8
MobileNet	16	70.4%	89.5%	4.3M	22.6	3.4
NASNetMobile	23	74.4%	91.9%	5.3M	27.0	6.7
EfficientNetB0	29	77.1%	93.3%	5.3M	46.0	4.9
EfficientNetB1	31	79.1%	94.4%	7.9M	60.2	5.6
EfficientNetB2	36	80.1%	94.9%	9.2M	80.8	6.5
EfficientNetB3	48	81.6%	95.7%	12.3M	140.0	8.8

EfficientNetB4	75	82.9%	96.4%	19.5M	308.3	15.1
InceptionV3	92	77.9%	93.7%	23.9M	42.2	6.9
ResNet50V2	98	76.0%	93.0%	25.6M	45.6	4.4
EfficientNetB5	118	83.6%	96.7%	30.6M	579.2	25.3
EfficientNetB6	166	84.0%	96.8%	43.3M	958.1	40.4
ResNet101V2	171	77.2%	93.8%	44.7M	72.7	5.4
InceptionResNetV2	215	80.3%	95.3%	55.9M	130.2	10.0
EfficientNetB7	256	84.3%	97.0%	66.7M	1578.9	61.6

I hope you enjoyed this deep dive into the main CNN architectures! Now let's see how to implement one of them using Keras.

Implementing a ResNet-34 CNN Using Keras

Most CNN architectures described so far can be implemented pretty naturally using Keras (although generally you would load a pretrained network instead, as we will see). To illustrate the process, let's implement a ResNet-34 from scratch using Keras. First, let's create a `ResidualUnit` layer:

```
DefaultConv2D = partial(tf.keras.layers.Conv2D, kernel_size=3, strides=1,
                        padding="same", kernel_initializer="he_normal",
                        use_bias=False)

class ResidualUnit(tf.keras.layers.Layer):
    def __init__(self, filters, strides=1, activation="relu", **kwargs):
        super().__init__(**kwargs)
        self.activation = tf.keras.activations.get(activation)
        self.main_layers = [
            DefaultConv2D(filters, strides=strides),
            tf.keras.layers.BatchNormalization(),
            self.activation,
            DefaultConv2D(filters),
            tf.keras.layers.BatchNormalization()
        ]
        self.skip_layers = []
        if strides > 1:
            self.skip_layers = [
                DefaultConv2D(filters, kernel_size=1, strides=strides),
                tf.keras.layers.BatchNormalization()
            ]

    def call(self, inputs):
        Z = inputs
        for layer in self.main_layers:
            Z = layer(Z)
        skip_Z = inputs
        for layer in self.skip_layers:
            skip_Z = layer(skip_Z)
        return self.activation(Z + skip_Z)
```

As you can see, this code matches [Figure 14-19](#) pretty closely. In the constructor, we create all the layers we will need: the main layers are the ones on the right side of the diagram, and the skip layers are the ones on the left (only needed if the stride is greater than 1). Then in the `call()` method, we make the inputs go through the main layers and the skip layers (if any), then we add both outputs and apply the activation function.

Next, we can build ResNet-34 using a `Sequential` model, since it's really just a long sequence of layers—we can treat each residual unit as a single layer now that we have the `ResidualUnit` class. The code closely matches [Figure 14-18](#):

```
model = tf.keras.Sequential([
    DefaultConv2D(64, kernel_size=7, strides=2, input_shape=[224, 224, 3]),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Activation("relu"),
    tf.keras.layers.MaxPool2D(pool_size=3, strides=2, padding="same"),
])
prev_filters = 64
for filters in [64] * 3 + [128] * 4 + [256] * 6 + [512] * 3:
    strides = 1 if filters == prev_filters else 2
    model.add(ResidualUnit(filters, strides=strides))
    prev_filters = filters

model.add(tf.keras.layers.GlobalAvgPool2D())
model.add(tf.keras.layers.Flatten())
model.add(tf.keras.layers.Dense(10, activation="softmax"))
```

The only tricky part in this code is the loop that adds the `ResidualUnit` layers to the model: as explained earlier, the first 3 RUs have 64 filters, then the next 4 RUs have 128 filters, and so on. At each iteration, we must set the stride to 1 when the number of filters is the same as in the previous RU, or else we set it to 2, then we add the `ResidualUnit`, and finally we update `prev_filters`.

It is amazing that in about 40 lines of code, we can build the model that won the ILSVRC 2015 challenge! This demonstrates both the elegance of the ResNet model and the expressiveness of the Keras API. Implementing the other CNN architectures is a bit longer, but not much harder. However, Keras comes with several of these architectures built in, so why not use them instead?

Using Pretrained Models from Keras

In general, you won't have to implement standard models like GoogLeNet or ResNet manually, since pretrained networks are readily available with a single line of code in the `tf.keras.applications` package. For example, you can load the ResNet-50 model, pretrained on ImageNet, with the following line of code:

```
model = tf.keras.applications.ResNet50(weights="imagenet")
```

That's all! This will create a ResNet-50 model and download weights pretrained on the ImageNet dataset. To use it, you first need to ensure that the images have the right size. A ResNet-50 model expects 224×224 -pixel images (other models may expect other sizes, such as 299×299), so let's use Keras's `Resizing` layer (introduced in [Chapter 13](#)) to resize two sample images (after cropping them to the target aspect ratio):

```
images = load_sample_images()["images"]
images_resized = tf.keras.layers.Resizing(height=224, width=224,
                                          crop_to_aspect_ratio=True)(images)
```

The pretrained models assume that the images are preprocessed in a specific way. In some cases they may expect the inputs to be scaled from 0 to 1, or from -1 to 1 , and so on. Each model provides a `preprocess_input()` function that you can use to preprocess your images. These functions assume that the original pixel values range from 0 to 255, which is the case here.

```
inputs = tf.keras.applications.resnet50.preprocess_input(images_resized)
```

Now we can use the pretrained model to make predictions:

```
>>> Y_proba = model.predict(inputs)
>>> Y_proba.shape
(2, 1000)
```

As usual, the output `Y_proba` is a matrix with one row per image and one column per class (in this case, there are 1,000 classes). If you want to display the top *K* predictions, including the class name and the estimated probability of each predicted class, use the `decode_predictions()` function. For each image, it returns an array containing the top *K* predictions, where each prediction is represented as an array containing the class identifier,²⁸ its name, and the corresponding confidence score:

```
top_K = tf.keras.applications.resnet50.decode_predictions(Y_proba, top=3)
for image_index in range(len(images)):
    print(f"Image #{image_index}")
    for class_id, name, y_proba in top_K[image_index]:
        print(f" {class_id} - {name:12s} {y_proba:.2%}")
```

The output looks like this:

```
Image #0
n03877845 - palace      54.69%
n03781244 - monastery   24.72%
n02825657 - bell_cote   18.55%
Image #1
n04522168 - vase        32.66%
n11939491 - daisy       17.81%
n03530642 - honeycomb   12.06%
```

The correct classes are *palace* and *dahlia*, so the model is correct for the first image but wrong for the second. However, that's because *dahlia* is not part of the 1,000 ImageNet classes. With that in mind, *vase* is a reasonable guess (perhaps the flower is in a vase?), and daisy is not a bad choice either, since dalias and daisies are both from the same Compositae family.

As you can see, it is very easy to create a pretty good image classifier using a pretrained model. As we saw in [Table 14-3](#), many other vision models are available in `tf.keras.applications`, from lightweight and fast models to large and accurate ones.

But what if you want to use an image classifier for classes of images that are not part of ImageNet? In that case, you may still benefit from the pretrained models to perform transfer learning.

Pretrained Models for Transfer Learning

If you want to build an image classifier but you do not have enough data to train it from scratch, then it is often a good idea to reuse the lower layers of a pretrained model, as we discussed in [Chapter 11](#). For example, let's train a model to classify pictures of flowers, reusing a pretrained Xception model. First, let's load the dataset using TensorFlow Datasets (introduced in [Chapter 13](#)):

```
import tensorflow_datasets as tfds

dataset, info = tfds.load("tf_flowers", as_supervised=True, with_info=True)
dataset_size = info.splits["train"].num_examples # 3670
class_names = info.features["label"].names # ["dandelion", "daisy", ...]
n_classes = info.features["label"].num_classes # 5
```

Note that you can get information about the dataset by setting `with_info=True`. Here, we get the dataset size and the names of the classes. Unfortunately, there is only a "train" dataset, no test set or validation set, so we need to split the training set. Let's call `tfds.load()` again, but this time taking the first 10% of the dataset for testing, the next 15% for validation, and the remaining 75% for training:

```
test_set_raw, valid_set_raw, train_set_raw = tfds.load(
    "tf_flowers",
    split=["train[:10%]", "train[10%:25%]", "train[25%:]"],
    as_supervised=True)
```

All three datasets contain individual images. We need to batch them, but for this we must first ensure they all have the same size, or else batching will fail. We can use a `Resizing` layer for this. We must also call the `tf.keras.applications.xception.preprocess_input()` function to preprocess the images appropriately for the Xception model. Lastly, let's also shuffle the training set and use prefetching:

```
batch_size = 32
preprocess = tf.keras.Sequential([
    tf.keras.layers.Resizing(height=224, width=224, crop_to_aspect_ratio=True),
    tf.keras.layers.Lambda(tf.keras.applications.xception.preprocess_input)
])
train_set = train_set_raw.map(lambda X, y: (preprocess(X), y))
train_set = train_set.shuffle(1000, seed=42).batch(batch_size).prefetch(1)
valid_set = valid_set_raw.map(lambda X, y: (preprocess(X), y)).batch(batch_size)
test_set = test_set_raw.map(lambda X, y: (preprocess(X), y)).batch(batch_size)
```

Now each batch contains 32 images, all of them 224×224 pixels, with pixel values ranging from -1 to 1 . Perfect!

Since the dataset is not very large, a bit of data augmentation will certainly help. Let's create a data augmentation model that we will embed in our final model: during training, it will randomly flip the images horizontally, rotate them a little bit, and tweak the contrast.

```
data_augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomFlip(mode="horizontal", seed=42),
    tf.keras.layers.RandomRotation(factor=0.05, seed=42),
    tf.keras.layers.RandomContrast(factor=0.2, seed=42)
])
```

TIP

The `tf.keras.preprocessing.image.ImageDataGenerator` class makes it easy to load images from disk and augment them in various ways: you can shift each image, rotate it, rescale it, flip it horizontally or vertically, shear it, or apply any transformation function you want to it. This is very convenient for simple projects. However, a `tf.data` pipeline is not much more complicated, and it's generally faster. Moreover, if you have a GPU and you include the preprocessing or data augmentation layers inside your model, they will benefit from GPU acceleration during training.

Next let's load an Xception model, pretrained on ImageNet. We exclude the top of the network by setting `include_top=False`. This excludes the global average pooling layer and the dense output layer. We then add our own global average pooling layer (feeding it the output of the base model), followed by a dense output layer with one unit per class, using the softmax activation function. Finally, we wrap all this in a Keras `Model`:

```
base_model = tf.keras.applications.xception.Xception(weights="imagenet",
    include_top=False)
avg = tf.keras.layers.GlobalAveragePooling2D()(base_model.output)
output = tf.keras.layers.Dense(n_classes, activation="softmax")(avg)
model = tf.keras.Model(inputs=base_model.input, outputs=output)
```

As explained in [Chapter 11](#), it's usually a good idea to freeze the weights of the pretrained layers, at least at the beginning of training:

```
for layer in base_model.layers:
    layer.trainable = False
```



```
model.compile(loss=["sparse_categorical_crossentropy", "mse"],
              loss_weights=[0.8, 0.2], # depends on what you care most about
              optimizer=optimizer, metrics=["accuracy"])
```

But now we have a problem: the flowers dataset does not have bounding boxes around the flowers. So, we need to add them ourselves. This is often one of the hardest and most costly parts of a Machine Learning project: getting the labels. It's a good idea to spend time looking for the right tools. To annotate images with bounding boxes, you may want to use an open source image labeling tool like VGG Image Annotator, LabelImg, OpenLabeler, or ImgLab, or perhaps a commercial tool like LabelBox or Supervisely. You may also want to consider crowdsourcing platforms such as Amazon Mechanical Turk if you have a very large number of images to annotate. However, it is quite a lot of work to set up a crowdsourcing platform, prepare the form to be sent to the workers, supervise them, and ensure that the quality of the bounding boxes they produce is good, so make sure it is worth the effort. Adriana Kovashka et al. wrote a very practical [paper](#)²⁹ about crowdsourcing in computer vision. I recommend you check it out, even if you do not plan to use crowdsourcing. If there are just a few hundred or a even a couple thousand images to label, and you don't plan to do this frequently, it may be preferable to do it yourself: with the right tools, it will only take a few days, and you'll also gain a better understanding of your dataset and task.

Now let's suppose you've obtained the bounding boxes for every image in the flowers dataset (for now we will assume there is a single bounding box per image). You then need to create a dataset whose items will be batches of preprocessed images along with their class labels and their bounding boxes. Each item should be a tuple of the form `(images, (class_labels, bounding_boxes))`. Then you are ready to train your model!

TIP

The bounding boxes should be normalized so that the horizontal and vertical coordinates, as well as the height and width, all range from 0 to 1. Also, it is common to predict the square root of the height and width rather than the height and width directly: this way, a 10-pixel error for a large bounding box will not be penalized as much as a 10-pixel error for a small bounding box.

The MSE often works fairly well as a cost function to train the model, but it is not a great metric to evaluate how well the model can predict bounding boxes. The most common metric for this is the *Intersection over Union* (IoU): the area of overlap between the predicted bounding box and the target bounding box, divided by the area of their union (see [Figure 14-24](#)). In Keras, it is implemented by the `tf.keras.metrics.MeanIoU` class.

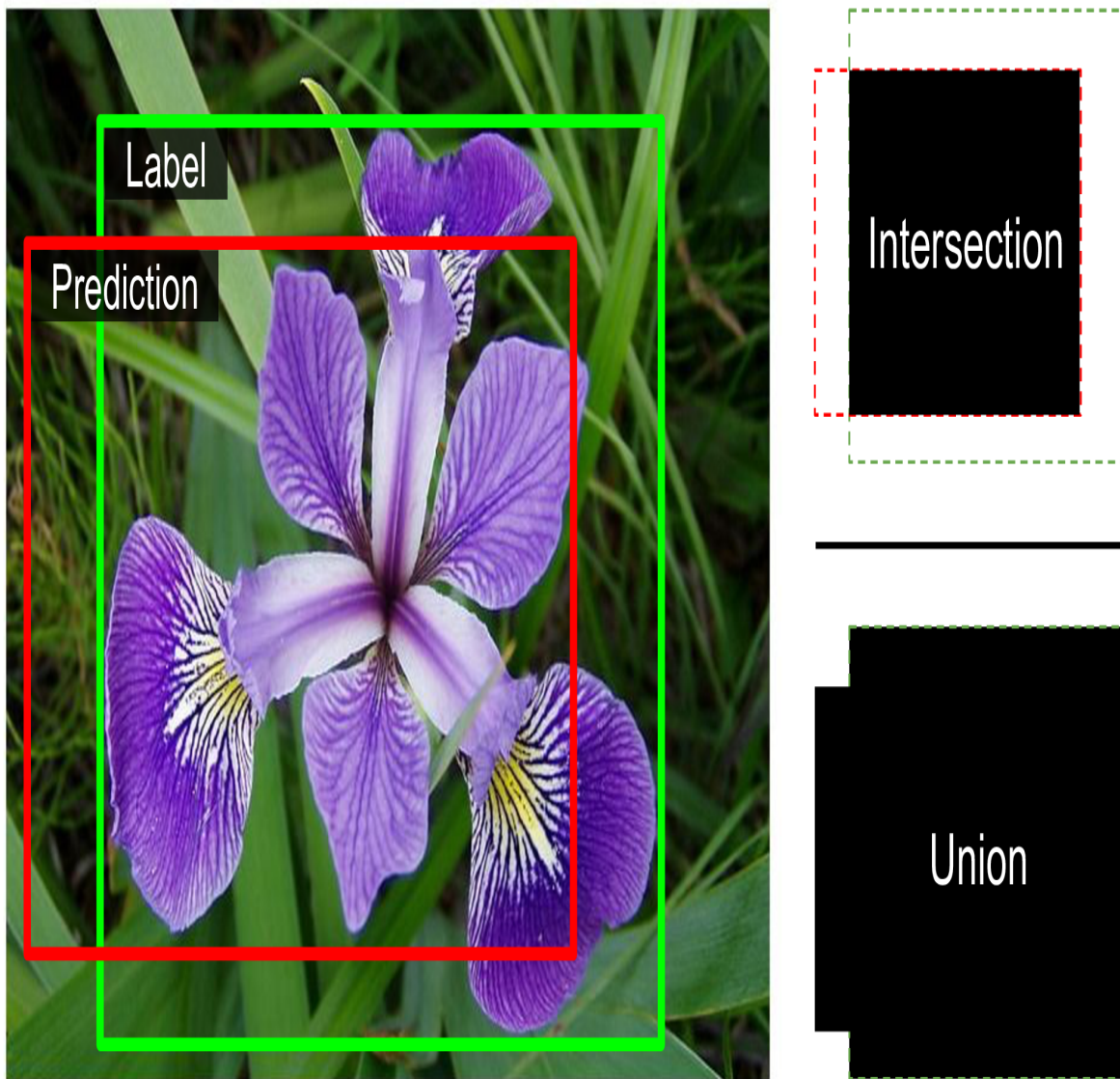


Figure 14-24. Intersection over Union (IoU) metric for bounding boxes

Classifying and localizing a single object is nice, but what if the images contain multiple objects (as is often the case in the flowers dataset)?

Object Detection

The task of classifying and localizing multiple objects in an image is called *object detection*. Until a few years ago, a common approach was to take a CNN that was trained to classify and locate a single object roughly centered in the image, then slide this CNN across the image, and make predictions at each step. The CNN was generally trained to predict not only class probabilities and a bounding box, but also an *objectness score*: this is the estimated probability that the image does indeed contain an object centered near the middle of the image. This is a binary classification output: it can be produced by a dense output layer with a single unit, using the sigmoid activation function and trained using the binary cross-entropy loss.

NOTE

Instead of an objectness score, a “no-object” class was sometimes added, but in general this did not work as well: the questions “is an object present?” and “what type of object is it?” are best answered separately.

This sliding-CNN approach is illustrated in [Figure 14-25](#). In this example, the image was chopped into a 5×7 grid, and we see a CNN—the thick black rectangle—sliding across all 3×3 regions, and making predictions at each step.

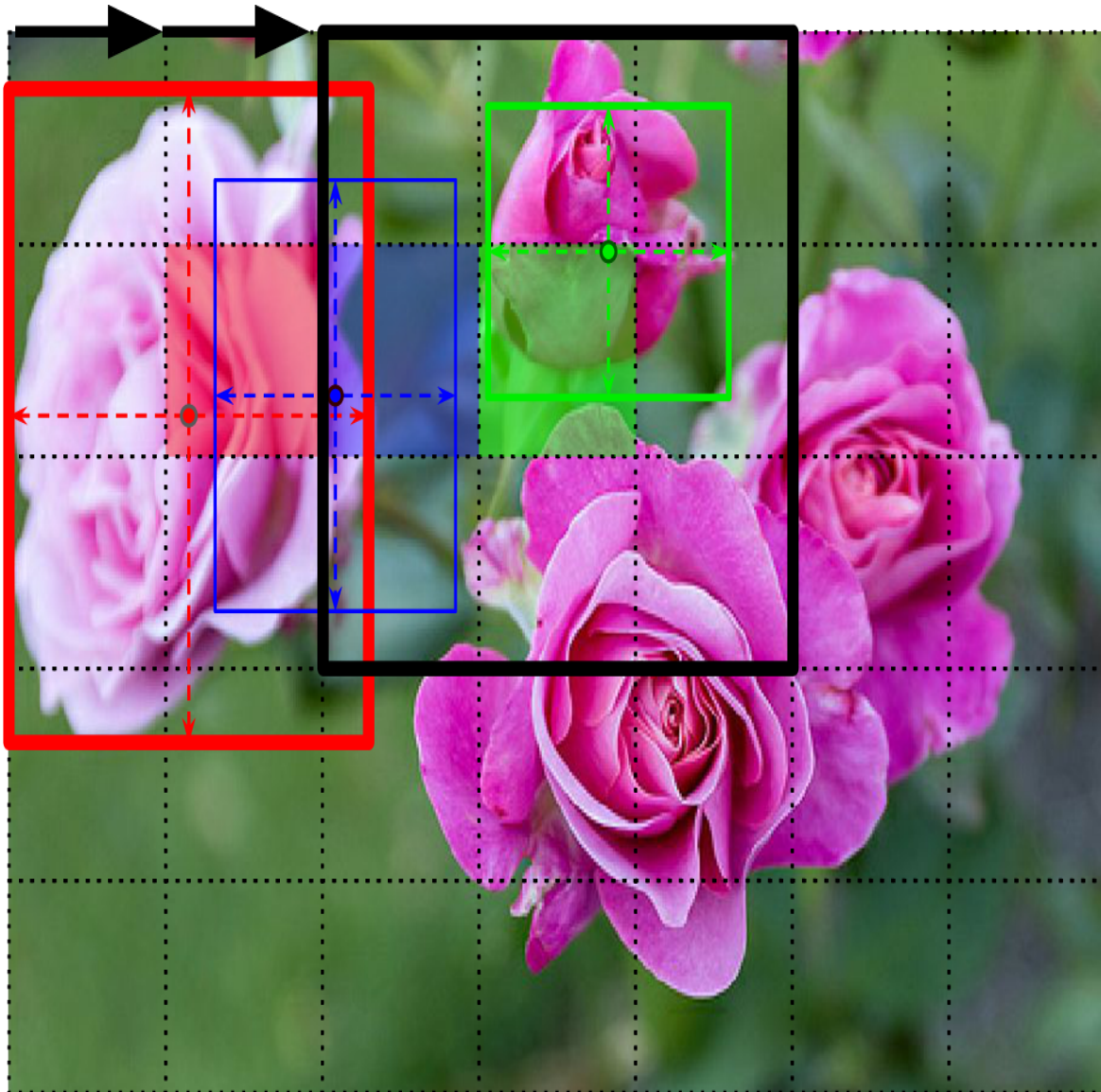


Figure 14-25. Detecting multiple objects by sliding a CNN across the image

In this figure, the CNN has already made predictions for three of these 3×3 regions:

- When looking at the top-left 3×3 region (centered on the red-shaded grid cell located in the second row and second column), it detected the leftmost rose. Notice that the predicted bounding box exceeds the boundary of this 3×3 region. That’s absolutely fine: even though the CNN could not see the bottom part of the rose, it was able to make a reasonable guess as to where it might be. It also predicted class probabilities, giving a high probability to the “rose” class. Lastly, it predicted a fairly high objectness score, since the center of the

bounding box lies within the central grid cell (in this figure, the objectness score is represented by the thickness of the bounding box).

- Then when looking at the next 3×3 region, one grid cell to the right (centered on the shaded blue square), it did not detect any flower centered in that region, so it predicted a very low objectness score, and therefore the predicted bounding box and class probabilities can safely be ignored. You can see that the predicted bounding box was no good anyway.
- Then, once again, it looked at the next 3×3 region, one grid cell to the right (centered on the shaded green cell), and this time it detected the rose at the top, although not perfectly since this rose is not well centered within this region, so the predicted objectness score was not very high.

You can imagine how sliding the CNN across the whole image would give you a total of 15 predicted bounding boxes, organized in a 3×5 grid, with each bounding box accompanied by its estimated class probabilities and objectness score. Since objects can have varying sizes, you may then want to slide the CNN again across larger 4×4 regions as well, to get even more bounding boxes.

This technique is fairly straightforward, but as you can see it will often detect the same object multiple times, at slightly different positions. Some post-processing is needed to get rid of all the unnecessary bounding boxes. A common approach for this is called *non-max suppression*. Here's how it works:

1. First, get rid of all the bounding boxes for which the objectness score is below some threshold: since the CNN believes there's no object at that location, the bounding box is useless.
2. Find the remaining bounding box with the highest objectness score, and get rid of all the other remaining bounding boxes that overlap a lot with it (e.g., with an IoU greater than 60%). For example, in [Figure 14-25](#), the bounding box with the max objectness score is the thick bounding box over the leftmost rose. The other bounding box that touches this same rose overlaps a lot with the max bounding box, so we will get rid of it (although in this example it would already have been removed in the previous step).
3. Repeat step two until there are no more bounding boxes to get rid of.

This simple approach to object detection works pretty well, but it requires running the CNN many times (15 times in this example), so it is quite slow. Fortunately, there is a much faster way to slide a CNN across an image: using a *fully convolutional network* (FCN).

Fully Convolutional Networks

The idea of FCNs was first introduced in a [2015 paper](#)³⁰ by Jonathan Long et al., for semantic segmentation (the task of classifying every pixel in an image according to the class of the object it belongs to). The authors pointed out that you could replace the dense layers at the top of a CNN, with convolutional layers. To understand this, let's look at an example: suppose a dense layer with 200 neurons sits on top of a convolutional layer that outputs 100 feature maps, each of size 7×7 (this is the feature map size, not the kernel size). Each neuron will compute a weighted sum of all $100 \times 7 \times 7$ activations from the convolutional layer (plus a bias term). Now let's see what happens if we replace the dense layer with a convolutional layer using 200 filters, each of size 7×7 , and with "valid" padding. This layer will output 200 feature maps, each 1×1 (since the kernel is exactly the size of the input feature maps and we are using "valid" padding). In other words, it will output 200 numbers, just like the dense layer did; and if you look closely at the computations performed by a convolutional layer, you will notice that these numbers will be precisely the same as those the dense layer produced. The only difference is that the dense layer's output was a tensor of shape `[batch size, 200]`, while the convolutional layer will output a tensor of shape `[batch size, 1, 1, 200]`.

TIP

To convert a dense layer to a convolutional layer, the number of filters in the convolutional layer must be equal to the number of units in the dense layer, the filter size must be equal to the size of the input feature maps, and you must use “valid” padding. The stride may be set to 1 or more, as we will see shortly.

Why is this important? Well, while a dense layer expects a specific input size (since it has one weight per input feature), a convolutional layer will happily process images of any size³¹ (however, it does expect its inputs to have a specific number of channels, since each kernel contains a different set of weights for each input channel). Since an FCN contains only convolutional layers (and pooling layers, which have the same property), it can be trained and executed on images of any size!

For example, suppose we’d already trained a CNN for flower classification and localization. It was trained on 224×224 images, and it outputs 10 numbers: outputs 0 to 4 are sent through the softmax activation function, and this gives the class probabilities (one per class); output 5 is sent through the sigmoid activation function, and this gives the objectness score; outputs 6 and 7 represent the bounding box’s center coordinates, and they also go through a sigmoid activation function to ensure they range from 0 to 1; lastly, outputs 8 and 9 represent the bounding box’s height and width, and they do not go through any activation function to allow the bounding boxes to extend beyond the borders of the image. We can now convert the CNN’s dense layers to convolutional layers. In fact, we don’t even need to retrain it; we can just copy the weights from the dense layers to the convolutional layers! Alternatively, we could have converted the CNN into an FCN before training.

Now suppose the last convolutional layer before the output layer (also called the bottleneck layer) outputs 7×7 feature maps when the network is fed a 224×224 image (see the left side of [Figure 14-26](#)). If we feed the FCN a 448×448 image (see the right side of [Figure 14-26](#)), the bottleneck layer will now output 14×14 feature maps.³² Since the dense output layer was replaced by a convolutional layer using 10 filters of size 7×7 , with “valid” padding and stride 1, the output will be composed of 10 feature maps, each of size 8×8 (since $14 - 7 + 1 = 8$). In other words, the FCN will process the whole image only once, and it will output an 8×8 grid where each cell contains 10 numbers (5 class probabilities, 1 objectness score, and 4 bounding box coordinates). It’s exactly like taking the original CNN and sliding it across the image using 8 steps per row and 8 steps per column. To visualize this, imagine chopping the original image into a 14×14 grid, then sliding a 7×7 window across this grid; there will be $8 \times 8 = 64$ possible locations for the window, hence 8×8 predictions. However, the FCN approach is *much* more efficient, since the network only looks at the image once. In fact, *You Only Look Once* (YOLO) is the name of a very popular object detection architecture, which we’ll look at next.

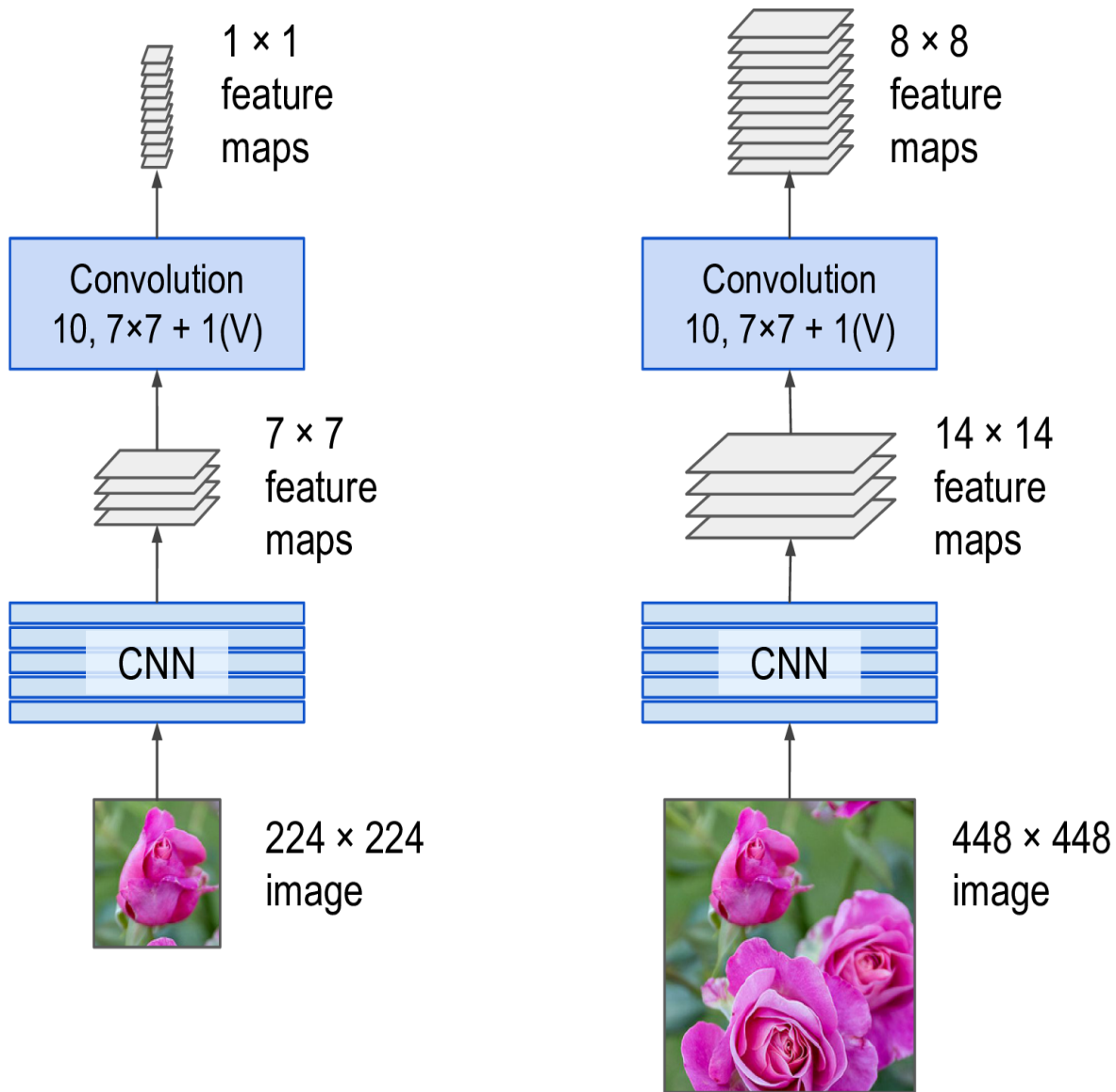


Figure 14-26. The same fully convolutional network processing a small image (left) and a large one (right)

You Only Look Once (YOLO)

YOLO is a fast and accurate object detection architecture proposed by Joseph Redmon et al. in a [2015 paper](#),³³. It is so fast that it can run in real time on a video, as seen in Redmon's [demo](#). YOLO's architecture is quite similar to the one we just discussed, but with a few important differences:

- For each grid cell, YOLO only considers objects whose bounding box center lies within that cell. The bounding box coordinates are relative to that cell, where (0, 0) means the top left of that cell and (1, 1) means the bottom right. However, the bounding box's height and width may extend well beyond the cell.
- It outputs two bounding boxes for each grid cell (instead of just one), which allows the model to handle cases where two objects are so close to each other that their bounding box centers lie within the same cell. Each bounding box also comes with its own objectness score.
- YOLO also outputs a class probability distribution for each grid cell, predicting 20 class probabilities per grid cell since YOLO was trained on the PASCAL VOC dataset, which contains 20 classes. This produces a coarse

class probability map. Note that the model predicts one class probability distribution per grid cell, not per bounding box. However, it's possible to estimate class probabilities for each bounding box during post-processing, by measuring how well each bounding box matches each class in the class probability map. For example, imagine a picture of a person standing in front of a car. There will be two bounding boxes: one large horizontal one for the car, and a smaller vertical one for the person. These bounding boxes may have their centers within the same grid cell. So how can we tell which class should be assigned to each bounding box? Well, the class probability map will contain a large region where the “car” class is dominant, and inside it there will be a smaller region where the “person” class is dominant. Hopefully, the car’s bounding box will roughly match the “car” region, while the person’s bounding box will roughly match the “person” region: this will allow the correct class to be assigned to each bounding box.

YOLO was originally developed using Darknet, an open source Deep Learning framework initially developed in C by Joseph Redmon, but it was soon ported to TensorFlow, Keras, PyTorch, and more. It was continuously improved over the years, with YOLOv2, YOLOv3 and YOLO9000 (again by Joseph Redmon et al.), YOLOv4 (by Alexey Bochkovskiy et al.), YOLOv5 (by Glenn Jocher), and PP-YOLO (by Xiang Long et al.). Each version brought some impressive improvements in speed and accuracy, using a variety of techniques. For example, YOLOv3 boosted accuracy in part thanks to *anchor priors*, exploiting the fact that some bounding box shapes are more likely than others, depending on the class (e.g., people tend to have vertical bounding boxes, while cars usually don’t). They also increased the number of bounding boxes per grid cell, they trained on different datasets with many more classes (up to 9,000 classes organized in a hierarchy in the case of YOLO9000), they added skip connections to recover some of the spatial resolution that is lost in the CNN (we will discuss this shortly, when we look at semantic segmentation), and much more. There are also many variants of these models, such as YOLOv4-tiny, which is optimized to be trained on less powerful machines, and which can run extremely fast (at over 1,000 frames per second!), but with a slightly lower *mean Average Precision* (mAP).

MEAN AVERAGE PRECISION (MAP)

A very common metric used in object detection tasks is the *mean Average Precision* (mAP). “Mean Average” sounds a bit redundant, doesn’t it? To understand this metric, let’s go back to two classification metrics we discussed in [Chapter 3](#): precision and recall. Remember the trade-off: the higher the recall, the lower the precision. You can visualize this in a precision/recall curve (see [Figure 3-6](#)). To summarize this curve into a single number, we could compute its area under the curve (AUC). But note that the precision/recall curve may contain a few sections where precision actually goes up when recall increases, especially at low recall values (you can see this at the top left of [Figure 3-6](#)). This is one of the motivations for the mAP metric.

Suppose the classifier has 90% precision at 10% recall, but 96% precision at 20% recall. There’s really no trade-off here: it simply makes more sense to use the classifier at 20% recall rather than at 10% recall, as you will get both higher recall and higher precision. So instead of looking at the precision *at* 10% recall, we should really be looking at the *maximum* precision that the classifier can offer with *at least* 10% recall. It would be 96%, not 90%. Therefore, one way to get a fair idea of the model’s performance is to compute the maximum precision you can get with at least 0% recall, then 10% recall, 20%, and so on up to 100%, and then calculate the mean of these maximum precisions. This is called the *Average Precision* (AP) metric. Now when there are more than two classes, we can compute the AP for each class, and then compute the mean AP (mAP). That’s it!

In an object detection system, there is an additional level of complexity: what if the system detected the correct class, but at the wrong location (i.e., the bounding box is completely off)? Surely we should not count this as a positive prediction. One approach is to define an IOU threshold: for example, we may consider that a prediction is correct only if the IOU is greater than, say, 0.5, and the predicted class is correct. The corresponding mAP is generally noted mAP@0.5 (or mAP@50%, or sometimes just AP₅₀). In some competitions (such as the PASCAL VOC challenge), this is what is done. In others (such as the COCO competition), the mAP is computed for different IOU thresholds (0.50, 0.55, 0.60, ..., 0.95), and the final metric is the mean of all these mAPs (noted mAP@[.50:.95] or mAP@[.50:0.05:.95]). Yes, that’s a mean mean average.

Many object detection models are available on TensorFlow Hub, often with pretrained weights, such as YOLOv5³⁴, SSD³⁵, Faster R-CNN,³⁶ or EfficientDet³⁷.

SSD and EfficientDet are “look once” detection models, similar to YOLO. EfficientDet is based on the EfficientNet convolutional architecture. Faster R-CNN is more complex: the image first goes through a CNN, then the output is passed to a *Region Proposal Network* (RPN) that proposes bounding boxes that are most likely to contain an object, and a classifier is then run for each bounding box, based on the cropped output of the CNN. The best place to start using these models is TensorFlow Hub’s excellent [object detection tutorial](#).

So far, we’ve only considered detecting objects in single images. But what about videos? Objects must not only be detected in each frame, they must also be tracked over time. Let’s take a quick look at object tracking now.

Object Tracking

Object tracking is a challenging task since objects move, they may grow or shrink as they get closer or further away from the camera, their appearance may change as they turn around or move to different lighting conditions or backgrounds, they may be temporarily occluded by other objects, and so on.

One of the most popular *object tracking* systems is DeepSORT³⁸. It is based on a combination of classical algorithms and Deep Learning:

- It uses *Kalman Filters* to estimate the most likely current position of an object given prior detections, and assuming that objects tend to move at a constant speed.
- It also uses a Deep Learning model to measure the resemblance between new detections and existing tracked objects.
- Lastly, it uses the *Hungarian algorithm* to map new detections to existing tracked objects (or to new tracked objects): this algorithm efficiently finds the combination of mappings that minimizes the distance between the detections and the predicted positions of tracked objects, while also minimizing the appearance discrepancy.

For example, imagine a red ball that just bounced on a blue ball. Based on the previous positions of the balls, the Kalman Filter will predict that the balls will go through each other: indeed, it assumes that objects move at a constant speed, so it will not expect the bounce. If the Hungarian algorithm only considered positions, then it would happily map the new detections to the wrong balls, as if they just went through each other and swapped colors. But thanks to the resemblance measure, the Hungarian algorithm will notice the problem. Assuming the balls are not too similar, the algorithm will map the new detections to the correct balls.

TIP

There are a few DeepSORT implementations available on GitHub. For example, this TensorFlow implementation of YOLOv4 + DeepSORT: <https://github.com/theAIGuysCode/yolov4-deepsort>.

So far we have located objects using bounding boxes. This is often sufficient, but sometimes you need to locate objects with much more precision, for example to remove the background behind a person during a videoconference call. Let’s see how to go down to the pixel level.

Semantic Segmentation

In *semantic segmentation*, each pixel is classified according to the class of the object it belongs to (e.g., road, car, pedestrian, building, etc.), as shown in [Figure 14-27](#). Note that different objects of the same class are *not* distinguished. For example, all the bicycles on the right side of the segmented image end up as one big lump of pixels. The main difficulty in this task is that when images go through a regular CNN, they gradually lose their

spatial resolution (due to the layers with strides greater than 1); so, a regular CNN may end up knowing that there's a person somewhere in the bottom left of the image, but it will not be much more precise than that.

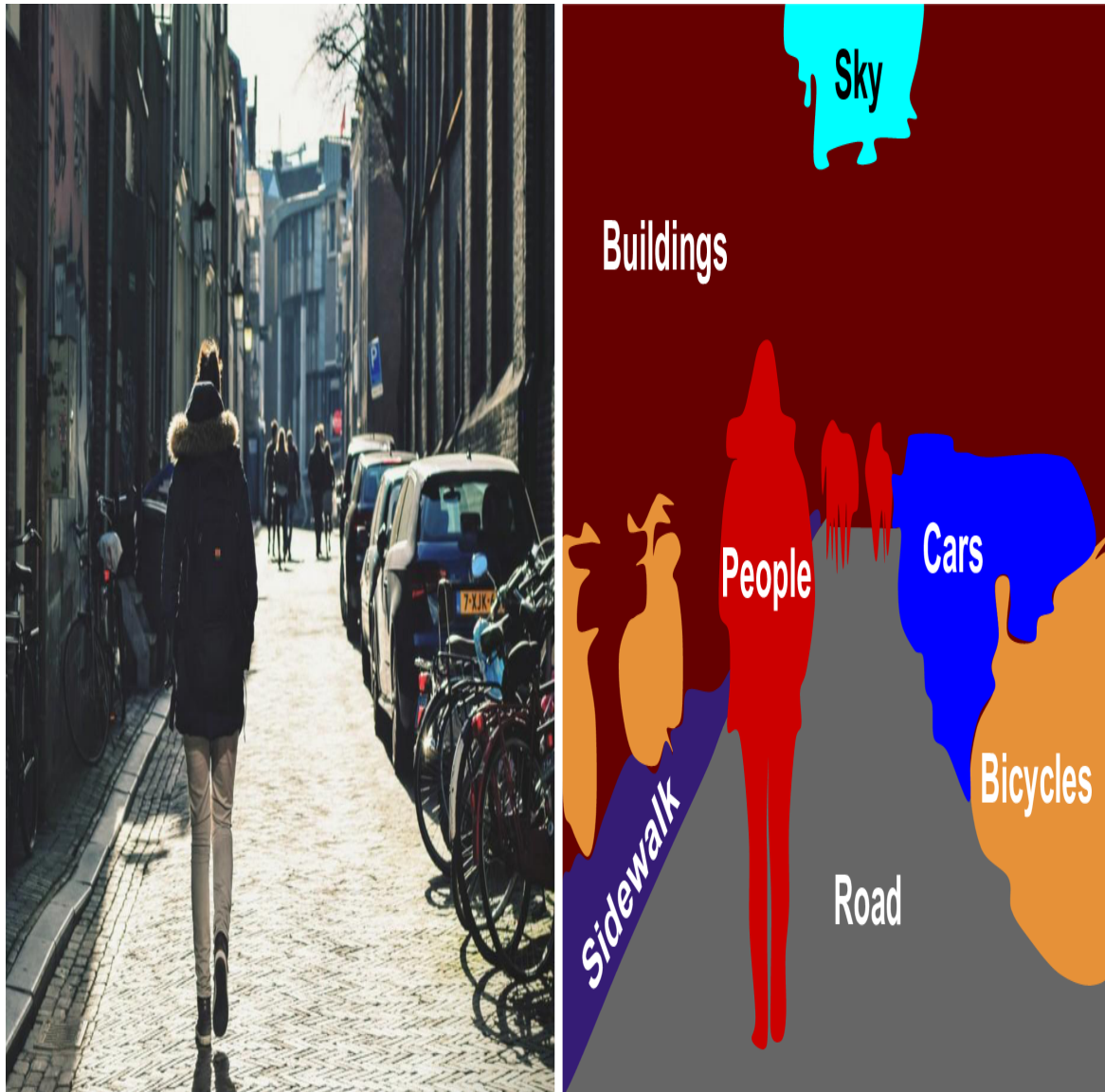


Figure 14-27. Semantic segmentation

Just like for object detection, there are many different approaches to tackle this problem, some quite complex. However, a fairly simple solution was proposed in the 2015 paper by Jonathan Long et al. we discussed earlier, on Fully Convolutional Networks. The authors start by taking a pretrained CNN and turning it into an FCN. The CNN applies an overall stride of 32 to the input image (i.e., if you add up all the strides greater than 1), meaning the last layer outputs feature maps that are 32 times smaller than the input image. This is clearly too coarse, so they add a single *upsampling layer* that multiplies the resolution by 32.

There are several solutions available for upsampling (increasing the size of an image), such as bilinear interpolation, but that only works reasonably well up to $\times 4$ or $\times 8$. Instead, they use a *transposed convolutional layer*:³⁹ it is equivalent to first stretching the image by inserting empty rows and columns (full of zeros), then performing a regular convolution (see Figure 14-28). Alternatively, some people prefer to think of it as a regular convolutional layer that uses fractional strides (e.g., $1/2$ in Figure 14-28). The transposed convolutional layer can

be initialized to perform something close to linear interpolation, but since it is a trainable layer, it will learn to do better during training. In Keras, you can use the Conv2DTranspose layer.

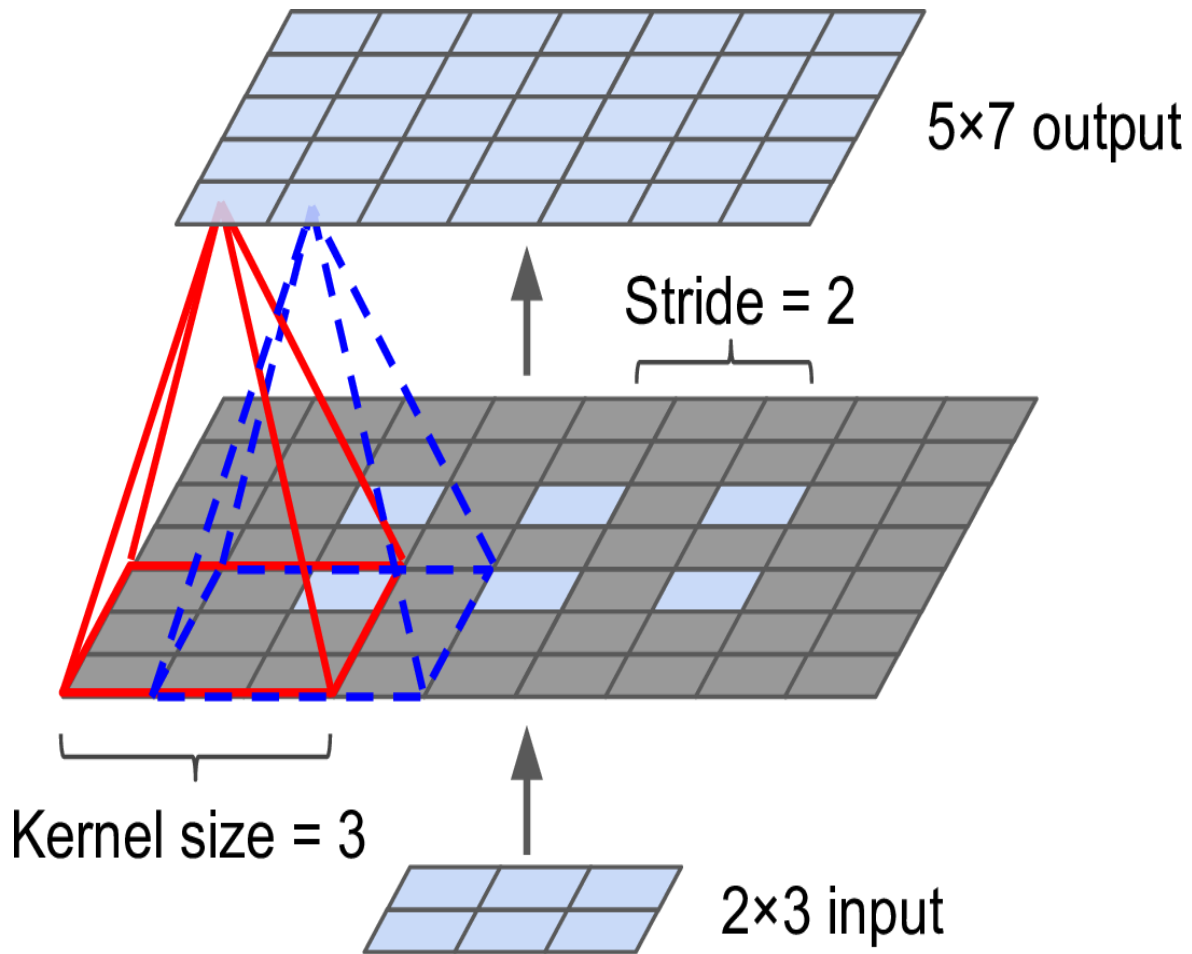


Figure 14-28. Upsampling using a transposed convolutional layer

NOTE

In a transposed convolutional layer, the stride defines how much the input will be stretched, not the size of the filter steps, so the larger the stride, the larger the output (unlike for convolutional layers or pooling layers).

OTHER KERAS CONVOLUTION LAYERS

Keras also offers a few other kinds of convolutional layers:

`tf.keras.layers.Conv1D`

A convolutional layer for 1D inputs, such as time series or text (sequences of letters or words), as we will see in [Chapter 15](#).

`tf.keras.layers.Conv3D`

A convolutional layer for 3D inputs, such as 3D PET scans.

`dilation_rate`

Setting the `dilation_rate` hyperparameter of any convolutional layer to a value of 2 or more creates an *à-trous convolutional layer* (“à trous” is French for “with holes”). This is equivalent to using a regular convolutional layer with a filter dilated by inserting rows and columns of zeros (i.e., holes). For example, a 1×3 filter equal to `[[1, 2, 3]]` may be dilated with a *dilation rate* of 4, resulting in a *dilated filter* of `[[1, 0, 0, 0, 2, 0, 0, 0, 3]]`. This lets the convolutional layer have a larger receptive field at no computational price and using no extra parameters.

Using transposed convolutional layers for upsampling is OK, but still too imprecise. To do better, the authors added skip connections from lower layers: for example, they upsampled the output image by a factor of 2 (instead of 32), and they added the output of a lower layer that had this double resolution. Then they upsampled the result by a factor of 16, leading to a total upsampling factor of 32 (see [Figure 14-29](#)). This recovered some of the spatial resolution that was lost in earlier pooling layers. In their best architecture, they used a second similar skip connection to recover even finer details from an even lower layer. In short, the output of the original CNN goes through the following extra steps: upsample $\times 2$, add the output of a lower layer (of the appropriate scale), upsample $\times 2$, add the output of an even lower layer, and finally upsample $\times 8$. It is even possible to scale up beyond the size of the original image: this can be used to increase the resolution of an image, which is a technique called *super-resolution*.

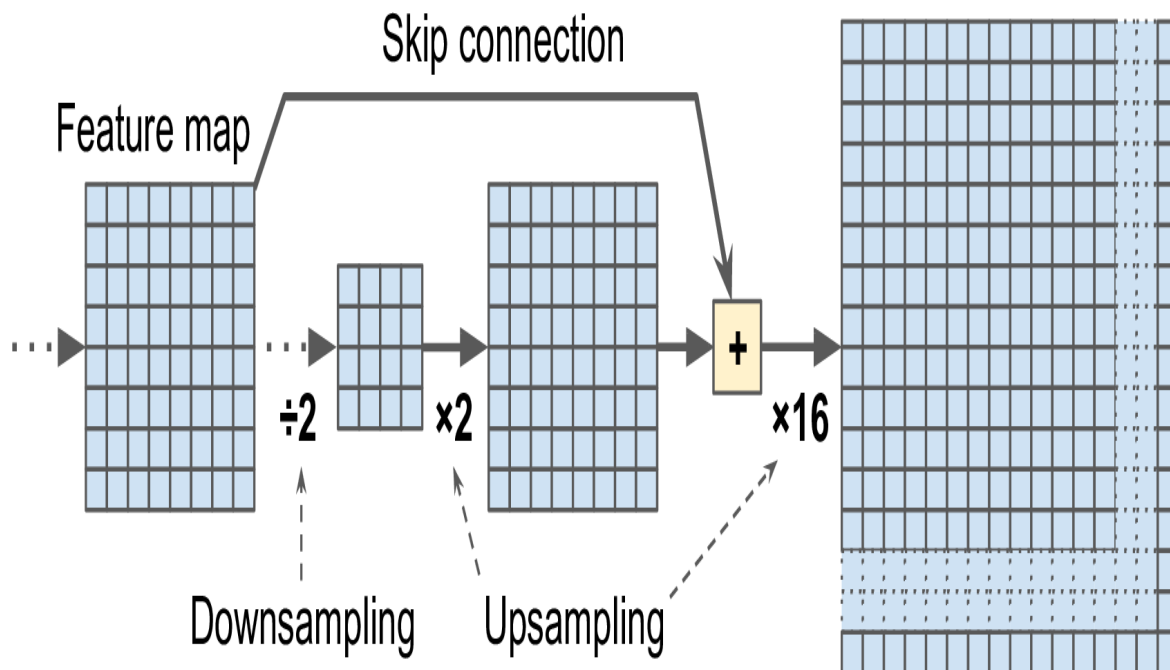


Figure 14-29. Skip layers recover some spatial resolution from lower layers

Instance segmentation is similar to semantic segmentation, but instead of merging all objects of the same class into one big lump, each object is distinguished from the others (e.g., it identifies each individual bicycle). For example the *Mask R-CNN* architecture, proposed in a [2017 paper](#)⁴⁰ by Kaiming He et al., extends the Faster R-CNN model by additionally producing a pixel mask for each bounding box. So not only do you get a bounding box around each object, with a set of estimated class probabilities, but you also get a pixel mask that locates pixels in the bounding box that belong to the object. This model is available on TensorFlow Hub, pretrained on the COCO 2017 dataset. But the field is moving fast, so if you want to try the latest and greatest models, please check out the state-of-the-art section on <https://paperswithcode.com/>.

As you can see, the field of Deep Computer Vision is vast and moving fast, with all sorts of architectures popping out every year. Almost all of them are based on convolutional neural networks, but since 2020 another neural net architecture has entered the computer vision space: Transformers (which we will discuss in [Chapter 16](#)). The progress made over the last decade has been astounding, and researchers are now focusing on harder and harder problems, such as *adversarial learning* (which attempts to make the network more resistant to images designed to fool it), *explainability* (understanding why the network makes a specific classification), realistic *image generation* (which we will come back to in [Chapter 17](#)), *single-shot learning* (a system that can recognize an object after it has seen it just once), predicting the next frames in a video, combining text and image tasks, and more.

Now on to the next chapter, where we will look at how to process sequential data such as time series using recurrent neural networks and convolutional neural networks.

Exercises

1. What are the advantages of a CNN over a fully connected DNN for image classification?
2. Consider a CNN composed of three convolutional layers, each with 3×3 kernels, a stride of 2, and “same” padding. The lowest layer outputs 100 feature maps, the middle one outputs 200, and the top one outputs 400. The input images are RGB images of 200×300 pixels.

What is the total number of parameters in the CNN? If we are using 32-bit floats, at least how much RAM will this network require when making a prediction for a single instance? What about when training on a mini-batch of 50 images?

3. If your GPU runs out of memory while training a CNN, what are five things you could try to solve the problem?
4. Why would you want to add a max pooling layer rather than a convolutional layer with the same stride?
5. When would you want to add a local response normalization layer?
6. Can you name the main innovations in AlexNet, compared to LeNet-5? What about the main innovations in GoogLeNet, ResNet, SEnet, Xception, and EfficientNet?
7. What is a fully convolutional network? How can you convert a dense layer into a convolutional layer?
8. What is the main technical difficulty of semantic segmentation?
9. Build your own CNN from scratch and try to achieve the highest possible accuracy on MNIST.
10. Use transfer learning for large image classification, going through these steps:
 - a. Create a training set containing at least 100 images per class. For example, you could classify your own pictures based on the location (beach, mountain, city, etc.), or alternatively you can use an existing dataset (e.g., from TensorFlow Datasets).
 - b. Split it into a training set, a validation set, and a test set.
 - c. Build the input pipeline, apply the appropriate preprocessing operations, and optionally add data augmentation.
 - d. Fine-tune a pretrained model on this dataset.
11. Go through TensorFlow's [Style Transfer tutorial](#). It is a fun way to generate art using Deep Learning.

Solutions to these exercises are available at the end of this chapter's notebook, at <https://homl.info/colab3>.

-
- 1 David H. Hubel, "Single Unit Activity in Striate Cortex of Unrestrained Cats," *The Journal of Physiology* 147 (1959): 226–238.
 - 2 David H. Hubel and Torsten N. Wiesel, "Receptive Fields of Single Neurons in the Cat's Striate Cortex," *The Journal of Physiology* 148 (1959): 574–591.
 - 3 David H. Hubel and Torsten N. Wiesel, "Receptive Fields and Functional Architecture of Monkey Striate Cortex," *The Journal of Physiology* 195 (1968): 215–243.
 - 4 Kunihiro Fukushima, "Neocognitron: A Self-Organizing Neural Network Model for a Mechanism of Pattern Recognition Unaffected by Shift in Position," *Biological Cybernetics* 36 (1980): 193–202.
 - 5 Yann LeCun et al., "Gradient-Based Learning Applied to Document Recognition," *Proceedings of the IEEE* 86, no. 11 (1998): 2278–2324.
 - 6 A convolution is a mathematical operation that slides one function over another and measures the integral of their pointwise multiplication. It has deep connections with the Fourier transform and the Laplace transform and is heavily used in signal processing. Convolutional layers actually use cross-correlations, which are very similar to convolutions (see <https://homl.info/76> for more details).
 - 7 To produce the same size outputs, a fully connected layer would need $200 \times 150 \times 100$ neurons, each connected to all $150 \times 100 \times 3$ inputs. It would have $200 \times 150 \times 100 \times (150 \times 100 \times 3 + 1) \approx 135$ billion parameters!
 - 8 In the international system of units (SI), $1 \text{ MB} = 1,000 \text{ KB} = 1,000 \times 1,000 \text{ bytes} = 1,000 \times 1,000 \times 8 \text{ bits}$. And $1 \text{ MiB} = 1,024 \text{ KiB} = 1,024 \times 1,024 \text{ bytes}$. So $12 \text{ MB} \approx 11.44 \text{ MiB}$.
 - 9 Other kernels we've discussed so far had weights, but pooling kernels do not: they are just stateless sliding windows.
 - 10 Yann LeCun et al., "Gradient-Based Learning Applied to Document Recognition," *Proceedings of the IEEE* 86, no. 11 (1998): 2278–2324.
 - 11 Alex Krizhevsky et al., "ImageNet Classification with Deep Convolutional Neural Networks," *Proceedings of the 25th International Conference on Neural Information Processing Systems* 1 (2012): 1097–1105.
 - 12 Matthew D. Zeiler and Rob Fergus, "Visualizing and Understanding Convolutional Networks," *Proceedings of the European Conference on Computer Vision* (2014): 818–833.
 - 13 Christian Szegedy et al., "Going Deeper with Convolutions," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (2015): 1–9.
 - 14 In the 2010 movie *Inception*, the characters keep going deeper and deeper into multiple layers of dreams; hence the name of these modules.

- 15 Karen Simonyan and Andrew Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” arXiv preprint arXiv:1409.1556 (2014).
- 16 Kaiming He et al., “Deep Residual Learning for Image Recognition,” arXiv preprint arXiv:1512.03385 (2015).
- 17 It is a common practice when describing a neural network to count only layers with parameters.
- 18 Christian Szegedy et al., “Inception–v4, Inception-ResNet and the Impact of Residual Connections on Learning,” arXiv preprint arXiv:1602.07261 (2016).
- 19 François Chollet, “Xception: Deep Learning with Depthwise Separable Convolutions,” arXiv preprint arXiv:1610.02357 (2016).
- 20 This name can sometimes be ambiguous, since spatially separable convolutions are often called “separable convolutions” as well.
- 21 Jie Hu et al., “Squeeze-and-Excitation Networks,” *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (2018): 7132–7141.
- 22 Saining Xie et al., “Aggregated Residual Transformations for Deep Neural Networks,” arXiv preprint arXiv:1611.05431 (2016).
- 23 Gao Huang et al., “Densely Connected Convolutional Networks,” arXiv preprint arXiv:1608.06993 (2016).
- 24 Andrew G. Howard et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” arXiv preprint arxiv:1704.04861 (2017).
- 25 Chien-Yao Wang et al., “CSPNet: A New Backbone that can Enhance Learning Capability of CNN,” arXiv preprint arXiv:1911.11929 (2019).
- 26 Mingxing Tan and Quoc V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” arXiv preprint arXiv:1905.11946 (2019).
- 27 A 92-core AMD EPYC CPU with IBPB, 1.7T of RAM, and an Nvidia Tesla A100 GPU.
- 28 In the ImageNet dataset, each image is mapped to a word in the **WordNet dataset**: the class ID is just a WordNet ID.
- 29 Adriana Kovashka et al., “Crowdsourcing in Computer Vision,” *Foundations and Trends in Computer Graphics and Vision* 10, no. 3 (2014): 177–243.
- 30 Jonathan Long et al., “Fully Convolutional Networks for Semantic Segmentation,” *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (2015): 3431–3440.
- 31 There is one small exception: a convolutional layer using “valid” padding will complain if the input size is smaller than the kernel size.
- 32 This assumes we used only “same” padding in the network: indeed, “valid” padding would reduce the size of the feature maps. Moreover, 448 can be neatly divided by 2 several times until we reach 7, without any rounding error. If any layer uses a different stride than 1 or 2, then there may be some rounding error, so again the feature maps may end up being smaller.
- 33 Joseph Redmon et al., “You Only Look Once: Unified, Real-Time Object Detection,” *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (2016): 779–788.
- 34 You can find YOLOv3, YOLOv4, and their tiny variants in the TensorFlow Models project at <https://homl.info/yolotf>.
- 35 Wei Liu et al., “SSD: Single Shot Multibox Detector,” *Proceedings of the 14th European Conference on Computer Vision* 1 (2016): 21–37.
- 36 Shaoqing Ren et al., “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks,” *Proceedings of the 28th International Conference on Neural Information Processing Systems* 1 (2015): 91–99.
- 37 Mingxing Tan et al., “EfficientDet: Scalable and Efficient Object Detection,” arXiv preprint arXiv:1911.09070 (2019).
- 38 Nicolai Wojke et al., “Simple Online and Realtime Tracking with a Deep Association Metric,” arXiv preprint arXiv:1703.07402 (2017).
- 39 This type of layer is sometimes referred to as a *deconvolution layer*, but it does *not* perform what mathematicians call a deconvolution, so this name should be avoided.
- 40 Kaiming He et al., “Mask R-CNN,” arXiv preprint arXiv:1703.06870 (2017).

Chapter 15. Processing Sequences Using RNNs and CNNs

With Early Release ebooks, you get books in their earliest form—the author’s raw and unedited content as they write—so you can take advantage of these technologies long before the official release of these titles.

This will be the 15th chapter of the final book. The GitHub repo is <https://github.com/ageron/handson-ml3>.

If you have comments about how we might improve the content and/or examples in this book, or if you notice missing material within this chapter, please reach out to the editor at mcronin@oreilly.com.

Predicting the future is something you do all the time, whether you are finishing a friend’s sentence or anticipating the smell of coffee at breakfast. In this chapter we will discuss recurrent neural networks (RNNs)—a class of nets that can predict the future (well, up to a point). RNNs can analyze time series data, such as the number of daily active users on your website, the hourly temperature in your city, your home’s daily power consumption, or the trajectories of nearby cars, and more. Once an RNN learns past patterns in the data, it is able to use its knowledge to forecast the future, assuming of course that past patterns still hold in the future.

More generally, RNNs can work on sequences of arbitrary lengths, rather than on fixed-sized inputs. For example, they can take sentences, documents, or audio samples as input, making them extremely useful for natural language processing applications such as automatic translation or speech-to-text.

In this chapter, we will first go through the fundamental concepts underlying RNNs and how to train them using backpropagation through time. Then, we will use them to forecast a time series. Along the way, we will look at the popular ARMA family of models, often used to forecast time series, and use them as baselines to compare with our RNNs. After that, we'll explore the two main difficulties that RNNs face:

- Unstable gradients (discussed in [Chapter 11](#)), which can be alleviated using various techniques, including *recurrent dropout* and *recurrent layer normalization*.
- A (very) limited short-term memory, which can be extended using LSTM and GRU cells.

RNNs are not the only types of neural networks capable of handling sequential data. For small sequences, a regular dense network can do the trick, and for very long sequences, such as audio samples or text, convolutional neural networks can actually work quite well too. We will discuss both of these possibilities, and we will finish this chapter by implementing a *WaveNet*—a CNN architecture capable of handling sequences of tens of thousands of time steps. Let's get started!

Recurrent Neurons and Layers

Up to now we have focused on feedforward neural networks, where the activations flow only in one direction, from the input layer to the output layer. A recurrent neural network looks very much like a feedforward neural network, except it also has connections pointing backward.

Let's look at the simplest possible RNN, composed of one neuron receiving inputs, producing an output, and sending that output back to itself, as shown in [Figure 15-1](#) (left). At each *time step* t (also called a *frame*), this *recurrent neuron* receives the inputs $\mathbf{x}_{(t)}$ as well as its own output from the previous time step, $\hat{y}_{(t-1)}$. Since there is no previous output at the first time step, it is generally set to 0. We can represent this tiny network against the time axis, as shown in [Figure 15-1](#) (right). This is called *unrolling the network*

through time (it's the same recurrent neuron represented once per time step).

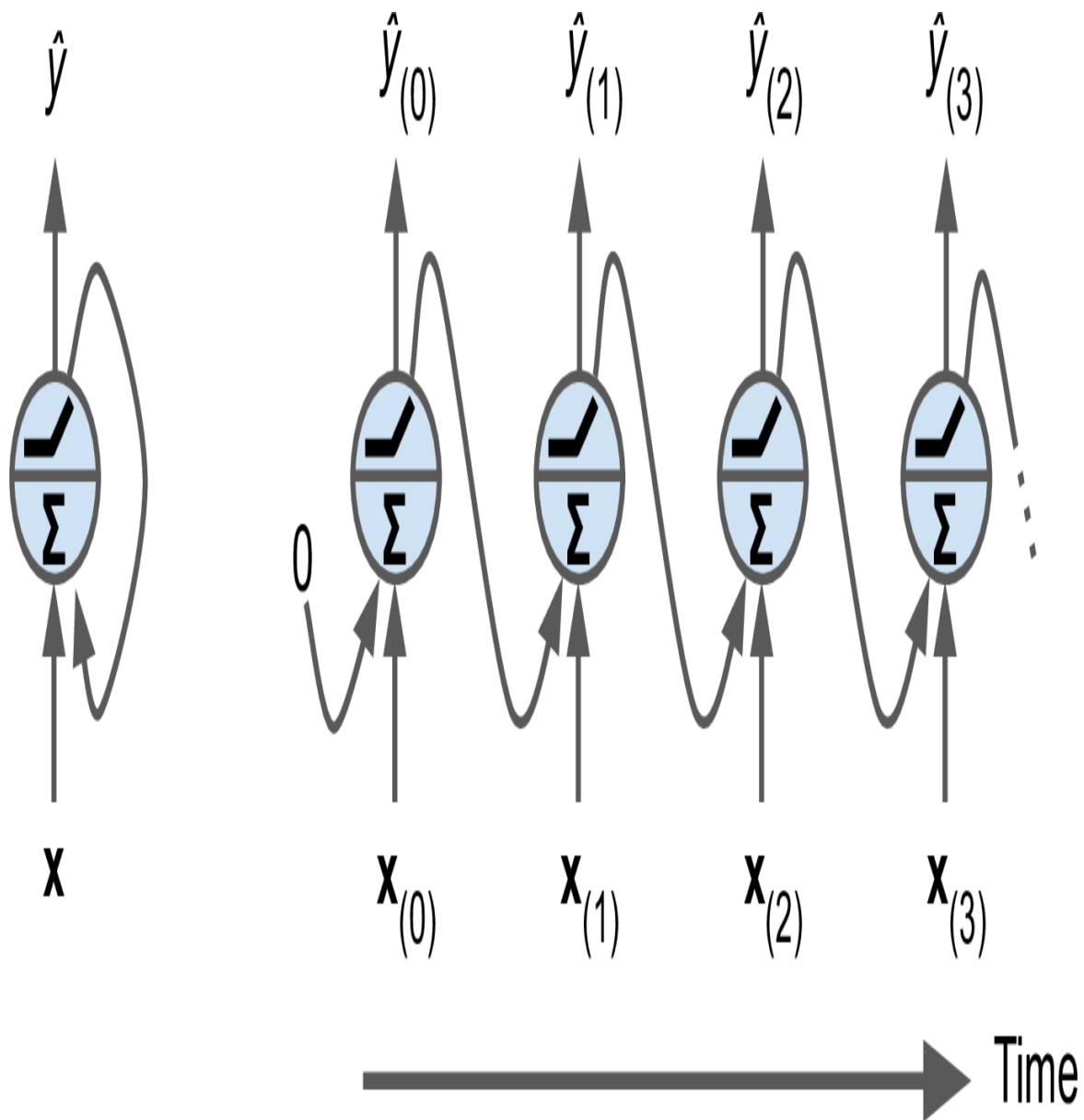


Figure 15-1. A recurrent neuron (left) unrolled through time (right)

You can easily create a layer of recurrent neurons. At each time step t , every neuron receives both the input vector $\mathbf{x}_{(t)}$ and the output vector from the previous time step $\hat{\mathbf{y}}_{(t-1)}$, as shown in [Figure 15-2](#). Note that both the inputs and outputs are now vectors (when there was just a single neuron, the output was a scalar).

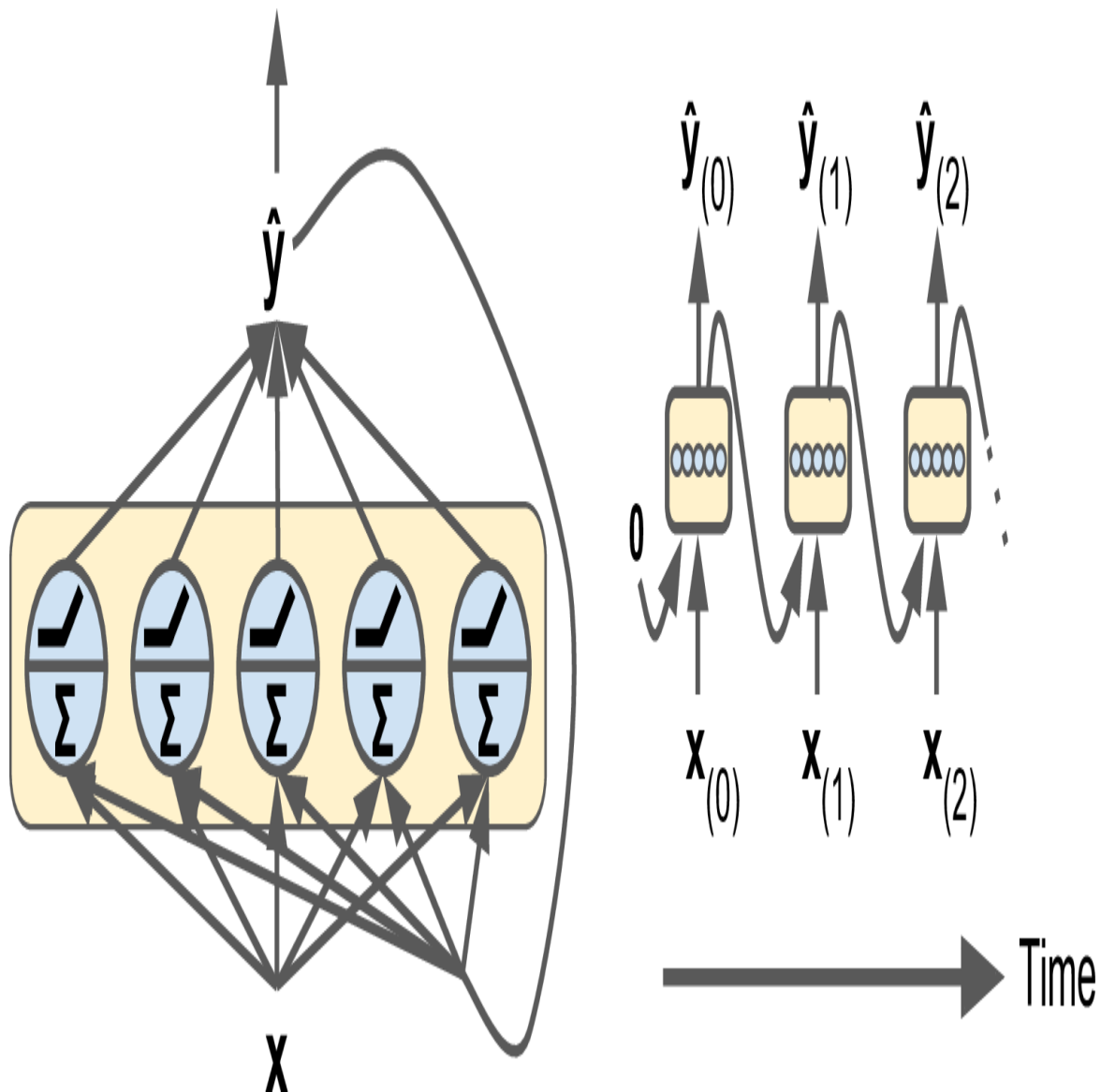


Figure 15-2. A layer of recurrent neurons (left) unrolled through time (right)

Each recurrent neuron has two sets of weights: one for the inputs $\mathbf{x}_{(t)}$ and the other for the outputs of the previous time step, $\hat{\mathbf{y}}_{(t-1)}$. Let's call these weight vectors $\mathbf{w}_x$ and $\mathbf{w}_{\hat{y}}$. If we consider the whole recurrent layer instead of just one recurrent neuron, we can place all the weight vectors in two weight matrices: $\mathbf{W}_x$ and $\mathbf{W}_{\hat{y}}$.

The output vector of the whole recurrent layer can then be computed pretty much as you might expect, as shown in [Equation 15-1](#), where $\mathbf{b}$ is the bias vector and $\phi(\cdot)$ is the activation function (e.g., ReLU¹).

Equation 15-1. Output of a recurrent layer for a single instance

$$\hat{\mathbf{y}}_{(t)} = \phi(\mathbf{W}_x^\top \mathbf{x}_{(t)} + \mathbf{W}_{\hat{\mathbf{y}}}^\top \hat{\mathbf{y}}_{(t-1)} + \mathbf{b})$$

Just as with feedforward neural networks, we can compute a recurrent layer's output in one shot for an entire mini-batch by placing all the inputs at time step t into an input matrix $\mathbf{X}_{(t)}$ (see [Equation 15-2](#)).

Equation 15-2. Outputs of a layer of recurrent neurons for all instances in a mini-batch

$$\begin{aligned}\hat{\mathbf{Y}}_{(t)} &= \phi(\mathbf{X}_{(t)} \mathbf{W}_x + \hat{\mathbf{Y}}_{(t-1)} \mathbf{W}_{\hat{\mathbf{y}}} + \mathbf{b}) \\ &= \phi\left(\begin{bmatrix} \mathbf{X}_{(t)} & \hat{\mathbf{Y}}_{(t-1)} \end{bmatrix} \mathbf{W} + \mathbf{b}\right) \text{ with } \mathbf{W} = \begin{bmatrix} \mathbf{W}_x \\ \mathbf{W}_{\hat{\mathbf{y}}} \end{bmatrix}\end{aligned}$$

In this equation:

- $\hat{\mathbf{Y}}_{(t)}$ is an $m \times n_{\text{neurons}}$ matrix containing the layer's outputs at time step t for each instance in the mini-batch (m is the number of instances in the mini-batch and n_{neurons} is the number of neurons).
- $\mathbf{X}_{(t)}$ is an $m \times n_{\text{inputs}}$ matrix containing the inputs for all instances (n_{inputs} is the number of input features).
- $\mathbf{W}_x$ is an $n_{\text{inputs}} \times n_{\text{neurons}}$ matrix containing the connection weights for the inputs of the current time step.
- $\mathbf{W}_{\hat{\mathbf{y}}}$ is an $n_{\text{neurons}} \times n_{\text{neurons}}$ matrix containing the connection weights for the outputs of the previous time step.
- $\mathbf{b}$ is a vector of size n_{neurons} containing each neuron's bias term.
- The weight matrices $\mathbf{W}_x$ and $\mathbf{W}_{\hat{\mathbf{y}}}$ are often concatenated vertically into a single weight matrix $\mathbf{W}$ of shape $(n_{\text{inputs}} + n_{\text{neurons}}) \times n_{\text{neurons}}$ (see the second line of [Equation 15-2](#)).
- The notation $\begin{bmatrix} \mathbf{X}_{(t)} & \hat{\mathbf{Y}}_{(t-1)} \end{bmatrix}$ represents the horizontal concatenation of the matrices $\mathbf{X}_{(t)}$ and $\hat{\mathbf{Y}}_{(t-1)}$.